CURSO 1 · AULA 4

Producers e Consumers na prática



Anatomia das APIs · Consumer Groups · Rebalance · Commit

Aula gravada · ~20 min · inclui demo (sandbox Docker)

O que vamos ver

01 Anatomia do Producer API — a linha de montagem do send()

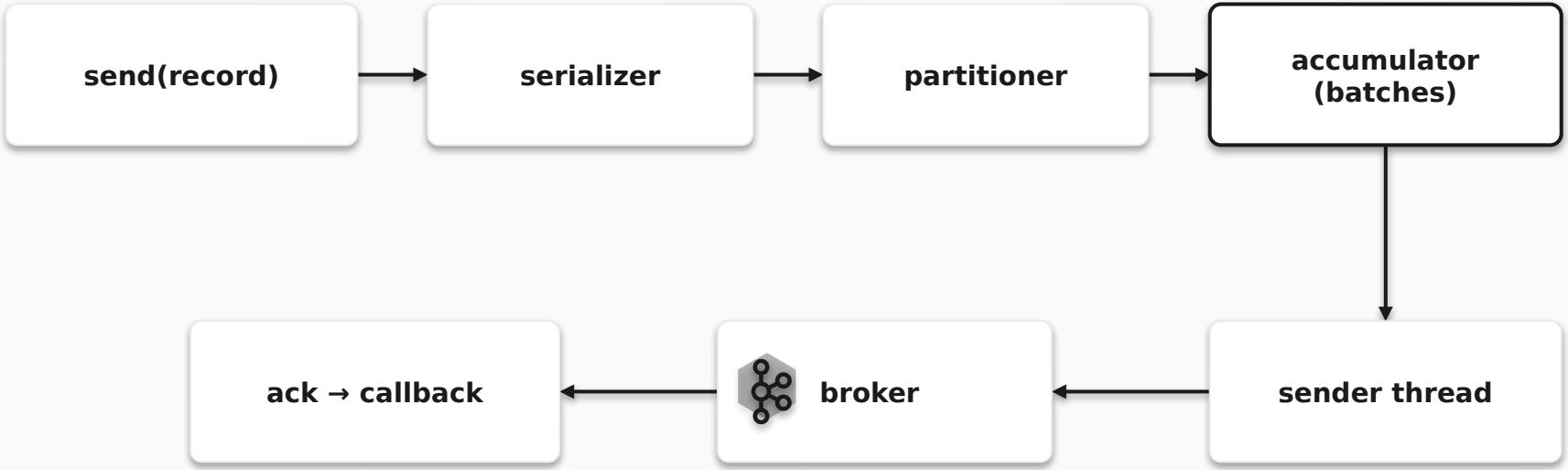
02 As configs que importam no producer

03 Anatomia do Consumer API — o poll loop

04 Consumer groups, rebalance e a regra de ouro

05 Commit de offset — a fronteira reprocessar × perder

Anatomia do Producer API



Producer: as configs que importam

acks

0 · 1 · all — a confirmação exigida. Peça central da Aula 5; default moderno = all.

enable.idempotence

Broker deduplica reenvios (default TRUE) — retry sem duplicar.

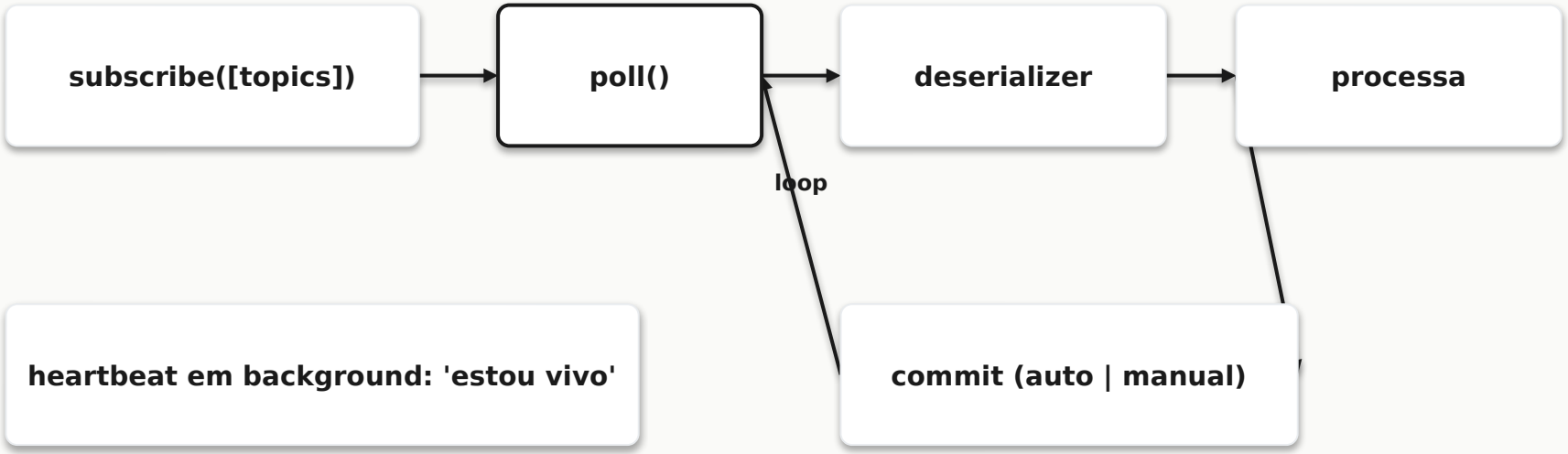
linger.ms + batch.size

Esperar um pouquinho pra mandar mais de uma vez — o trade-off latência × throughput.

compression.type

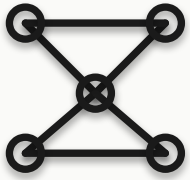
lz4/zstd: menos rede e disco ao custo de CPU — quase sempre vale.

Anatomia do Consumer API: o poll loop



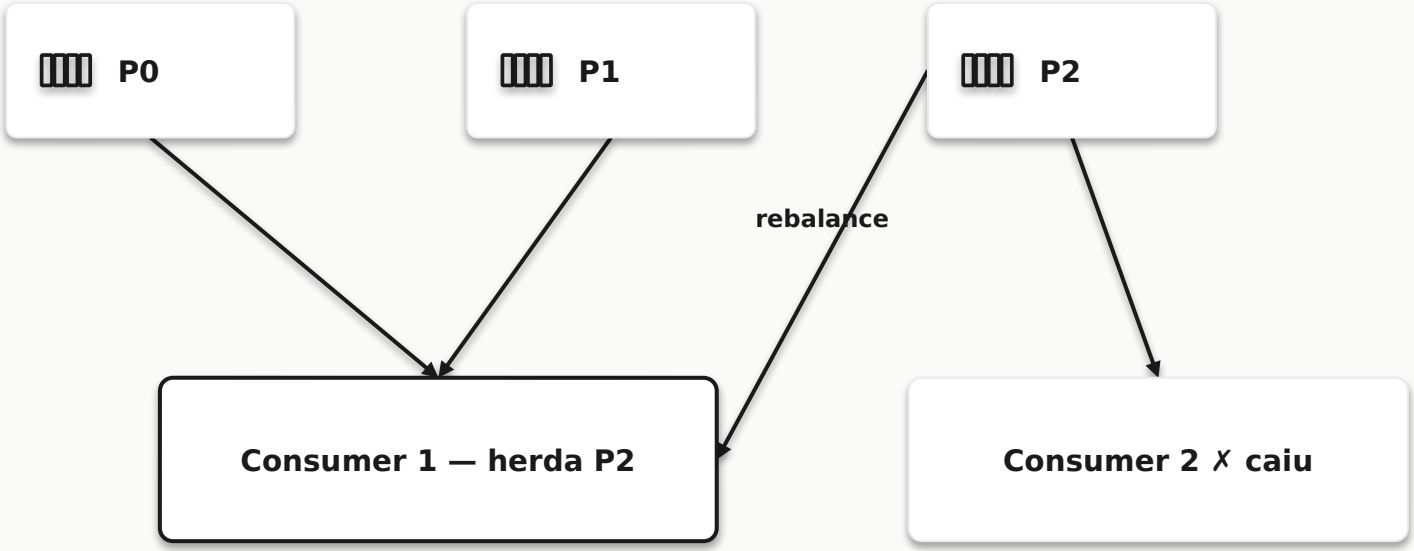
Consumer group: a regra de ouro

Cada partição é lida por no máximo UM consumidor do grupo.



- 3 partições + 3 consumidores → paralelismo máximo
- 3 partições + 5 consumidores → 2 ficam OCIOSOS
- Partições são o TETO do paralelismo (Aula 3 cobrando)

Rebalance: o grupo se reorganiza



Commit: a fronteira reprocessar × perder

O grupo registra até onde leu, por partição. QUANDO confirmar define o que acontece na falha.



- Confirmar cedo demais → risco de PERDER
- Confirmar após processar → pode RELER (at-least-once)
- Auto-commit = simples com janelas; manual = controle

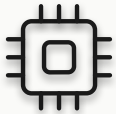
Demo: rebalance ao vivo

Producer contínuo (1 msg/s) no tópico de 3 partições. Cenas:

```
# 1 consumer sozinho no grupo → atende as 3 partições
kafka-console-consumer --group pedidos-app ...
# entra o 2º no MESMO grupo → REBALANCE na tela, fluxo se divide
# Ctrl+C num deles → o sobrevivente HERDA as partições
kafka-consumer-groups --describe --group pedidos-app
#   → CURRENT-OFFSET · LOG-END-OFFSET · LAG por partição
```

Esperado: O grupo escala, se recupera e registra progresso — e o LAG é consultável (no Curso 7, vira o principal alerta de produção).

PONTO DE REFLEXÃO



Seu consumer processa cada lote em 8 min — e max.poll.interval.ms está em 5 min. O que acontece com o grupo?

Pause: qual o sintoma, e quais DUAS saídas você tem?

Recap

- send() é assíncrono e em batch — serializer → partitioner → accumulator → sender
- Consumer vive no poll loop — e poll é sinal de vida
- Regra de ouro: 1 partição → máx 1 consumidor do grupo
- Rebalance dá elasticidade e tolerância a falha (com custo)
- Commit é a fronteira entre reprocessar e perder

O grupo escala e se recupera — mas quem herdou pode ter relido.

A seguir → Aula 5 — Ordem por Chave, Idempotência e Semânticas de Entrega